

09 | 初识trait：协议约束与能力配置

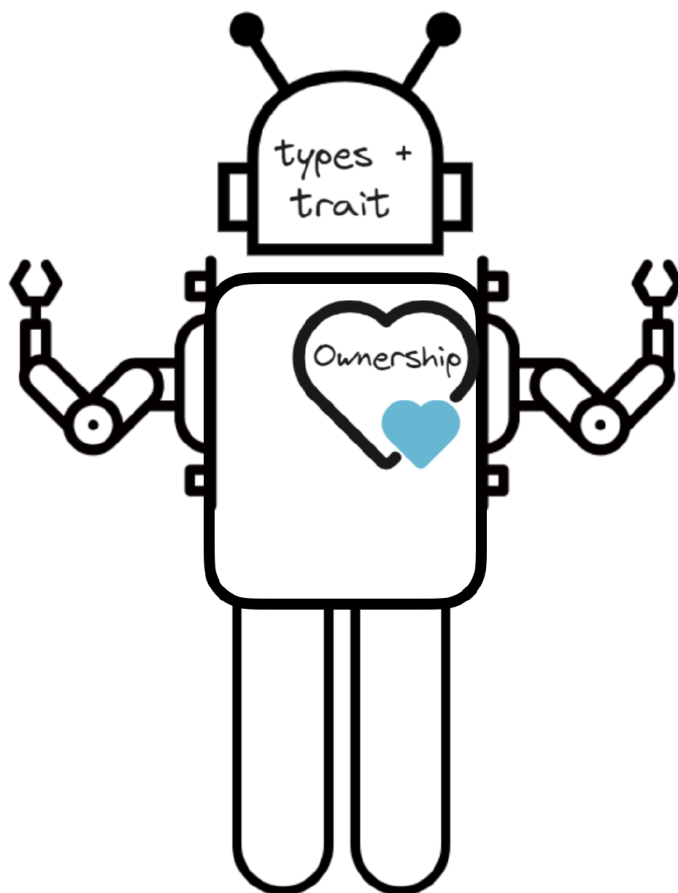
唐刚 · Rust 语言从入门到实战



你好，我是 Mike。今天我们来一起学习 trait。

trait 在 Rust 中非常重要。如果说所有权是 Rust 中的九阳神功（内功护体），那么类型系统（types + trait）就是 Rust 中的降龙十八掌，学好了便如摧枯拉朽般解决问题。另外一方面，如果把 Rust 比作一个 AI 人的话，那么所有权相当于 Rust 的心脏，类型 + trait 相当于这个 AI 人的大脑。

Rust



好了，我就不卖关子了。前面我们已经学习了所有权，现在我们来了解一下这个 trait 到底是什么。

注：所有权是这门课程的主线，会一直贯穿到最后。

trait 是什么？

trait 用中文来讲就是特征，但是我倾向于不翻译。因为 trait 本身很简单，它就是一个标记（marker 或 tag）而已。比如 `trait TraitA {}` 就定义了一个 trait，TraitA。

只不过这个标记被用在特定的地方，也就是类型参数的后面，用来限定（bound）这个类型参数可能的类型范围。所以 trait 往往是跟类型参数结合起来使用的。比如 `T: TraitA` 就是使用 TraitA 对类型参数 T 进行限制。

这么讲起来比较抽象，我们下面举例说明。

trait 是一种约束

我们先回忆一下 [第 7 讲](#) 的一个例子。

[复制代码](#)

```
1 struct Point<T> {
2     x: T,
3     y: T,
4 }
5
6 fn print<T: std::fmt::Display>(p: Point<T>) {
7     println!("Point {}, {}", p.x, p.y);
8 }
9
10 fn main() {
11     let p = Point {x: 10, y: 20};
12     print(p);
13
14     let p = Point {x: 10.2, y: 20.4};
15     print(p);
16 }
17 // 输出
18 Point 10, 20
19 Point 10.2, 20.4
```

注意代码里的第六行。

[复制代码](#)

```
1 fn print<T: std::fmt::Display>(p: Point<T>) {
```

这里的 `Display` 就是一个 trait，用来对类型参数 `T` 进行约束。它表示**必须要实现了 `Display` 的类型才能被代入类型参数 `T`**，也就是限定了 `T` 可能的类型范围。`std::fmt::Display` 这个 trait 主要是定义成配合格式化参数 `"{}"` 使用的，和它相对的还有 `std::fmt::Debug`，用来定义成配合格式化参数 `"{:?}"` 使用。而例子里的整数和浮点数，都默认实现了这个 `Display` trait，因此整数和浮点数这两种类型能够代入函数 `print()` 的类型参数 `T`，从而执行打印的功能。

我们看一下如果一个类型没有实现 Display，把它代入 print() 函数，会发生什么。

[复制代码](#)

```

1 struct Point<T> {
2     x: T,
3     y: T,
4 }
5
6 struct Foo; // 新定义了一种类型
7
8 fn print<T: std::fmt::Display>(p: Point<T>) {
9     println!("Point {}, {}", p.x, p.y);
10 }
11
12 fn main() {
13     let p = Point {x: 10, y: 20};
14     print(p);
15
16     let p = Point {x: 10.2, y: 20.4};
17     print(p);
18
19     let p = Point {x: Foo, y: Foo}; // 初始化一个Point<T> 实例
20     print(p);
21 }

```

报编译错误：

[复制代码](#)

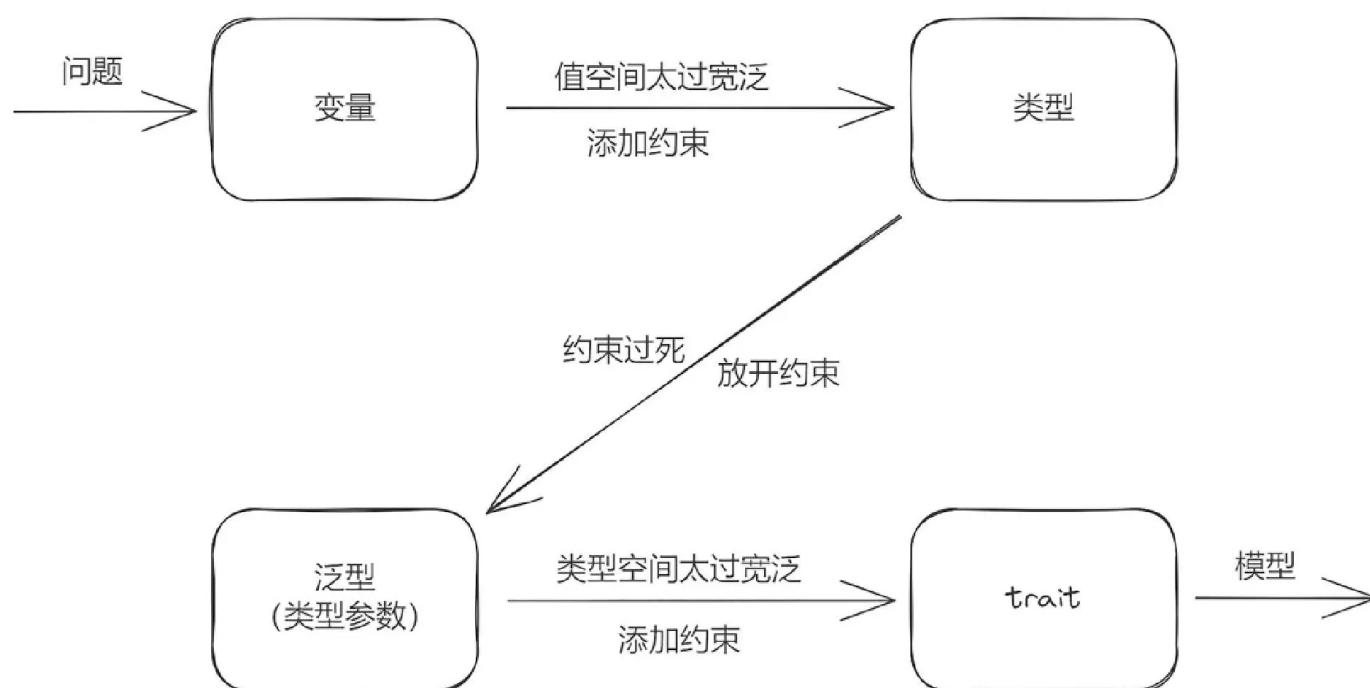
```

1 error[E0277]: `Foo` doesn't implement `std::fmt::Display`
2   --> src/main.rs:20:11
3   |
4 20 |     print(p);
5   |         ^ `Foo` cannot be formatted with the default formatter
6   |
7   |     required by a bound introduced by this call
8   |
9   = help: the trait `std::fmt::Display` is not implemented for `Foo`
10  = note: in format strings you may be able to use `{:?}` (or `{:#?}` for pretty-p

```

提示说，Foo 类型没有实现 Display，所以没办法编译通过。

回顾一下我们前面 [第 7 讲](#) 里说到的：类型是对变量值空间的约束。结合 trait 是对类型参数类型空间的约束。我们可以整理出一张图来表达它们之间的关系。



首先，如果我们定义一个变量，这个时候还没有给它指定类型。那么这个时候这个变量的取值空间就是任意的。由于值空间太过宽泛，我们给它指定类型，比如 `u32` 或字符串，给这个变量的值空间添加了约束。并且指定了明确的类型后，**这个变量的值空间就被限定为仅这一种类型的值空间。**

对于简单的应用，到这一步也够了，但对抽象程度比较高的复杂应用，这种限定就显得太死了。所以又引入了类型参数（泛型）这个机制，用一个类型参数 `T` 就可以代表不同的类型。这样，以往为一些特定类型开发的函数等，就可以在不改变逻辑的情况下，扩展到支持多种类型，这样就灵活了很多。

但是，只有一个类型参数 `T`，这个信息还不够，对同一个函数，可能只有少数一些类型适用，而符号 `T` 本身如果不加任何限制的话，它的内涵（类型空间）就太宽泛了，它可以取任意类型。所以光引入类型参数这个机制还不够，还得配套地引入一个对类型参数进行约束的机制。于是就出现了 `trait`。结合图片，我们可以明白引入 `trait` 的原因和 `trait` 起的作用。

另一方面，从图片里其实也可以看出用 Rust 对问题建模的思维方式。当你在头脑中经过这样一个过程后，针对一个问题在 Rust 中就能建立起对应的模型来，整个过程非常自然。输入问题，输出模型！

语法上，`T: TraitA` 意思就是我们对类型参数 `T` 施加了 `TraitA` 这个约束标记。那么，我们怎么给某种具体的类型实现 `TraitA`，来让那个具体的类型可以代入 `T` 呢？具体来说，我们要对某个类型 `Atype` 实现某个 trait 的话，使用语法 `impl TraitA for Atype {}` 就可以做到。

只需要下面三行代码就可以。

 复制代码

```
1 trait TraitA {}
2
3 struct Atype;
4
5 impl TraitA for Atype {}
```

这三行代码是可以通过编译的，是不是有点吃惊？

对于某个类型 `T`（这里指的是某种具体的类型）来说，如果它实现了这个 `TraitA`，我们就说这个类型**满足约束**。

 复制代码

```
1 T: TraitA
```

一个 trait 在一个类型上只能被实现一次。比如：

 复制代码

```
1 trait TraitA {}
2
3 struct Atype;
4
5 impl TraitA for Atype {}
```

```
6 impl TraitA for Atype {}  
7 // 输出, 编译错误:  
8 error[E0119]: conflicting implementations of trait `TraitA` for type `Atype`
```

例子也很好理解，约束声明一次就够了，多次声明就冲突了，不知道哪一个生效。

trait 是一种能力配置

如果 trait 仅仅是一个纯标记名称，而不包含内容的话，那它的作用是非常有限的。

```
1 trait TraitA {}
```

[复制代码](#)

下面我们会知道，这个{}里可以放入一些元素，这些元素属于这个 trait。

我们先接着前面那个示例继续讲。

```
1 fn print<T: std::fmt::Display>(p: Point<T>) {
```

[复制代码](#)

Display 对类型参数 T 作了约束，要求将来要代入的具体类型必须实现了 Display 这个 trait。另一方面，也可以说成，Display 给将来要代入到这个类型参数里的具体类型提供了一套“能力”，这套能力是在 Display 这个 trait 中定义和封装的。具体来说，就是能够打印的能力，因为确实有些值是没法打印出来的，比如原始二进制编码，打出来也是乱码。而 Display 就提供了打印的能力，同时还定义了具体的打印要求。

注：Display 是标准库提供的一种常用 trait，我们会在第 11 讲专门讲解标准库里的各种常用 trait。

也就是说，**trait 对类型参数实施约束的同时，也对具体的类型提供了能力**。让我们看到类型参数后面的约束，就知道到时候代入这其中的类型会具有哪些能力。比如我们看到了

Display，就知道那些类型具有打印的能力。我们看到了 PartialEq，就知道那些类型具有比较大小的能力等等。

你也可以这样理解，在 Rust 中**约束和能力就是一体两面，是同一个东西**。这样下面的写法是不是就好理解多了？

```
1 T: TraitA + TraitB + TraitC + TraitD
```

[复制代码](#)

这个约束表达式，给某种类型 T 提供了从 TraitA 到 TraitD 这 4 套能力。我们后面还会看到，基于多 trait 组合的约束表达式，可以用这种方式提供优美的能力（权限）配置。

那么，一个 trait 里面具体可以有哪些东西呢？

trait 中包含什么？

trait 里面可以包含关联函数、关联类型和关联常量。

关联函数

在 trait 里可以定义关联函数。比如下面 Sport 这个 trait 就定义了四个关联函数。

```
1 trait Sport {  
2     fn play(&self);      // 注意这里直接以分号结尾，表示函数签名  
3     fn play_mut(&mut self);  
4     fn play_own(self);  
5     fn play_some() -> Self;  
6 }
```

[复制代码](#)

例子里，前 3 个关联函数都带有 Self 参数（⚠️ 所有权三态又出现了），它们被实现到具体类型上的时候，就成为那个具体类型的方法。第 4 个方法，play_some() 函数里第一个参数不是 Self 类型，也就是说，它不是 self、&self、&mut self 中的一个，它被实现在具体类型上的时候，就是那个类型的关联函数。

可以看到，在 trait 中可以使用 Rust 语言里的标准类型 Self，用来指代将要被实现这个 trait 的那个类型。使用 impl 语法将一个 trait 实现到目标类型上去。

你可以看一下示例。

 复制代码

```
1 struct Football;
2
3 impl Sport for Football {
4     fn play(&self) {} // 注意函数后面的花括号，表示实现
5     fn play_mut(&mut self) {}
6     fn play_own(self) {}
7     fn play_some() -> Self { Self }
8 }
```

这里这个 Self，就指代 Football 这个类型。

trait 中也可以定义关联函数的默认实现。

比如：

 复制代码

```
1 trait Sport {
2     fn play(&self) {} // 注意这里一对花括号，就是trait的关联函数的默认实现
3     fn play_mut(&mut self) {}
4     fn play_own(self); // 注意这里是以分号结尾，就表示没有默认实现
5     fn play_some() -> Self;
6 }
```

在这个示例里，play() 和 play_mut() 后面定义了函数体，因此实际上提供了默认实现。

有了 trait 关联函数的默认实现后，具体类型在实现这个 trait 的时候，就可以“偷懒”，直接利用默认实现。比如：

 复制代码

```

1 struct Football;
2
3
4 impl Sport for Football {
5     fn play_own(self) {}
6     fn play_some() -> Self { Self }
7 }

```

这个跟下面这个例子效果是一样的。

[复制代码](#)

```

1 struct Football;
2
3 impl Sport for Football {
4     fn play(&self) {}
5     fn play_mut(&mut self) {}
6     fn play_own(self) {}
7     fn play_some() -> Self { Self }
8 }

```

上面的代码相当于 `Football` 类型重新实现了一次 `play()` 和 `play_mut()` 函数，覆盖了 `trait` 的这两个函数的默认实现。

在类型上实现了 `trait` 后就可以使用这些方法了。

[复制代码](#)

```

1 fn main () {
2     let mut f = Football;
3     f.play();          // 方法在实例上调用
4     f.play_mut();
5     f.play_own();
6     let _g = Football::play_some();    // 关联函数要在类型上调用
7     let _g = <Football as Sport>::play_some(); // 注意这样也是可以的
8 }

```

关联类型

在 `trait` 中，可以带一个或多个关联类型。关联类型起一种类型占位功能，定义 `trait` 时声明，在把 `trait` 实现到类型上的时候为其指定具体的类型。比如：

```
1 pub trait Sport {
2     type SportType;
3
4     fn play(&self, st: Self::SportType);
5 }
6
7 struct Football;
8 pub enum SportType {
9     Land,
10    Water,
11 }
12
13 impl Sport for Football {
14     type SportType = SportType; // 这里故意取相同的名字，不同的名字也是可以的
15     fn play(&self, st: Self::SportType){} // 方法中用到了关联类型
16 }
17
18 fn main() {
19     let f = Football;
20     f.play(SportType::Land);
21 }
```

解释一下，我们在给 Football 类型实现 Sport trait 的时候，指明具体的关联类型 SportType 为一个枚举类型，用来区分陆地运动与水上运动。注意看 trait 中的 play 方法的第二个参数，它就是用的关联类型占位。

在 T 上使用关联类型

我们再来看一个示例，标准库中迭代器 Iterator trait 的定义。

```
1 pub trait Iterator {
2     type Item;
3
4     fn next(&mut self) -> Option<Self::Item>;
5 }
```


Iterator 定义了一个关联类型 Item。注意这里的 `Self::Item` 实际是 `<Self as Iterator>::Item` 的简写。一般来说，如果一个类型参数被 TraitA 约束，而 TraitA 里有关联类型 MyType，那么可以用 `T::Mytype` 这种形式来表示路由到这个关联类型。

比如：

 复制代码

```
1 trait TraitA {
2     type Mytype;
3 }
4
5 fn doit<T: TraitA>(a: T::Mytype) {} // 这里在函数中使用了关联类型
6
7 struct TypeA;
8 impl TraitA for TypeA {
9     type Mytype = String; // 具化关联类型为String
10 }
11
12 fn main() {
13     doit::<TypeA>("abc".to_string()); // 给Rustc小助手喂信息：T具化为TypeA
14 }
```

上面示例在 `doit()` 函数中使用了 TraitA 中的关联类型，用的是 `T::Mytype` 这种路由 / 路径形式。在 `main()` 函数中调用 `doit()` 函数时，手动把类型参数 T 具化为 TypeA。你可以多花一些时间熟悉一下这种表达形式。

在约束中具化关联类型

在指定约束的时候，可以把关联类型具化。你可以看一下我给出的示例。

 复制代码

```
1 trait TraitA {
2     type Item;
3 }
4 struct Foo<T: TraitA<Item=String>> { // 这里在约束表达式中对关联类型做了具化
5     x: T
6 }
7 struct A;
8 impl TraitA for A {
```

```
9     type Item = String;
10 }
11
12 fn main() {
13     let a = Foo {
14         x: A,
15     };
16 }
```

上面的代码在约束表达式中对关联类型做了具化，具化为 String 类型。

[复制代码](#)

```
1 T: TraitA<Item=String>
```

这样表达的意思就是限制必须实现了 TraitA，而且它的关联类型必须是 String 才能代入这个 T。假如我们稍微改一下，把类型 A 实现 TraitA 时的关联类型 Item 具化为 u32，就会编译报错，你可以试着编译一下看下提示。

[复制代码](#)

```
1 trait TraitA {
2     type Item;
3 }
4 struct Foo<T: TraitA<Item=String>> {
5     x: T
6 }
7 struct A;
8 impl TraitA for A {
9     type Item = u32; // 这里类型不匹配
10 }
11
12 fn main() {
13     let a = Foo {
14         x: A, // 报错
15     };
16 }
```

慢慢地我们给的示例有些烧脑了，现在你并不需要精通这些用法，**第一步是要认识它们**，当你看到别人写这种代码的时候，能基本看懂就可以了。

对关联类型的约束

在定义关联类型的时候，也可以给关联类型添加约束。意思是后面在具化这个类型的时候，那些类型必须要满足于这些约束，或者说实现过这些约束。你可以看我给出的这个例子。

[复制代码](#)

```
1 use std::fmt::Debug;
2
3 trait TraitA {
4     type Item: Debug; // 这里对关联类型添加了Debug约束
5 }
6
7 #[derive(Debug)] // 这里在类型A上自动derive Debug约束
8 struct A;
9
10 struct B;
11
12 impl TraitA for B {
13     type Item = A; // 这里这个类型A已满足Debug约束
14 }
```

在使用时甚至可以加强对关联类型的约束。比如：

[复制代码](#)

```
1 use std::fmt::Debug;
2
3 trait TraitA {
4     type Item: Debug; // 这里对关联类型添加了Debug约束
5 }
6
7 #[derive(Debug)]
8 struct A;
9
10 struct B;
11
12 impl TraitA for B {
13     type Item = A; // 这里这个类型A已满足Debug约束
14 }
15
16 fn doit<T>() // 定义类型参数T
17 where
18     T: TraitA, // 使用where语句将T的约束表达放在后面来
19     T::Item: Debug + PartialEq // 注意这一句，直接对TraitA的关联类型Item添加了更多一个约束
```

```
20 {  
21 }
```

请注意上面例子里的 `doit()` 函数。我们使用 `where` 语句把类型参数 `T` 的约束表达放在后面，同时使用 `T::Item: Debug + PartialEq` 来加强对 `TraitA` 的关联类型 `Item` 的约束，表示只有实现过 `TraitA` 且其关联类型 `Item` 的具化版必须满足 `Debug` 和 `PartialEq` 的约束。

这个例子稍微有点复杂，不过理解后，你会感觉到 Rust trait 的精髓。目前你可以把这个示例当作思维体操来练一练，没事了回来细品一下。另外，你可以自己修改这个例子，看看 Rustc 小助手会告诉你什么。

关联常量

同样的，trait 里也可以携带一些常量信息，表示这个 trait 的一些内在信息（挂载在 trait 上的信息）。和关联类型不同的是，关联常量可以在 trait 定义的时候指定，也可以在给具体的类型实现的时候指定。

你可以看一下这个例子。

[复制代码](#)

```
1 trait TraitA {  
2     const LEN: u32 = 10;  
3 }  
4  
5 struct A;  
6 impl TraitA for A {  
7     const LEN: u32 = 12;  
8 }  
9  
10 fn main() {  
11     println!("{:?}", A::LEN);  
12     println!("{:?}", <A as TraitA>::LEN);  
13 }  
14 //输出  
15 12  
16 12
```

如果在 impl 的时候不指定，会有什么效果呢？你可以看看代码运行后的结果。

 复制代码

```
1 trait TraitA {  
2     const LEN: u32 = 10;  
3 }  
4  
5 struct A;  
6 impl TraitA for A {}  
7  
8 fn main() {  
9     println!("{:?}", A::LEN);  
10    println!("{:?}", <A as TraitA>::LEN);  
11 }  
12 //输出  
13 10  
14 10
```

trait 作为一种协议

我们已经看到，trait 里有可选的关联函数、关联类型、关联常量这三项内容。一旦 trait 定义好，它就相当于一法律或协议，在实现它的各个类型之间，在团队协作中不同的开发者之间，都必须按照它定义的规范实施。这是**强制性的**，而且这种强制性是由 Rust 编译器来执行的。也就是说，**如果你不想按这套协议来实施，那么你注定无法编译通过。**

这个对于团队开发来说非常重要。它相当于在团队中协调的接口协议，强制不同成员之间达成一致。**从这个意义上来讲，Rust 非常适合团队开发。**

Where

当类型参数后面有多个 trait 约束的时候，会显得“头重脚轻”，比较难看，所以 Rust 提供了 Where 语法来解决这个问题。Where 关键字可用来把约束关系统一放在后面表示。

比如这个函数：

 复制代码

```
1 fn doit<T: TraitA + TraitB + TraitC + TraitD + TraitE>(t: T) -> i32 {}
```

这行代码可以写成下面这种形式。

 复制代码

```
1 fn doit<T>(t: T) -> i32
2 where
3     T: TraitA + TraitB + TraitC + TraitD + TraitE
4 {}
```

这样过多的 trait 约束就不至于太干扰函数签名的视觉完整性。

约束依赖

Rust 还提供了一种语法表示约束间的依赖。

 复制代码

```
1 trait TraitA: TraitB {}
```

初看起来，这跟 C++ 等语言的类的继承有点像。实际不是，差异很大。**这个语法的意思是如果某种类型要实现 TraitA，那么它也要同时实现 TraitB。**反过来不成立。

例子：

 复制代码

```
1 trait Shape { fn area(&self) -> f64; }
2 trait Circle : Shape { fn radius(&self) -> f64; }
```

上面这两行代码其实等价于下面这两行代码。

```
1 trait Shape { fn area(&self) -> f64; }
2 trait Circle where Self: Shape { fn radius(&self) -> f64; }
```

 复制代码

你也可以看一下使用时的约束表示。

```
1 T: Circle
2 实际上表示：
3 T: Circle + Shape
```

 复制代码

在这个约束依赖的限定下，如果你对一个类型实现了 Circle trait，却没有实现 Shape，那么 Rust 小助手会提示你这个类型不满足约束 Shape。

比如下面代码：

```
1 trait Shape {}
2 trait Circle : Shape {}
3 struct A;
4 struct B;
5 impl Shape for A {}
6 impl Circle for A {}
7 impl Circle for B {}
```

 复制代码

提示出错：

```
1 error[E0277]: the trait bound `B: Shape` is not satisfied
2   --> src/main.rs:7:17
3   |
4 7 | impl Circle for B {}
5   |                   ^ the trait `Shape` is not implemented for `B`
6   |
7   = help: the trait `Shape` is implemented for `A`
8 note: required by a bound in `Circle`
9   --> src/main.rs:2:16
10  |
```

 复制代码

```

11 2 | trait Circle : Shape {}
12   |           ^^^^^ required by this bound in `Circle`
13

```

一个 trait 依赖多个 trait 也是可以的。

[复制代码](#)

```
1 trait TraitA: TraitB + TraitC {}
```

这个例子里面，`T: TraitA` 实际表 `T: TraitA + TraitB + TraitC`。因此可以少写不少代码。

约束之间是完全平等的，理解这一点非常重要，通过刚刚的这些例子可以看到约束依赖是消除约束条件冗余的一种方式。

在约束依赖中，冒号后面的叫 supertrait，冒号前面的叫 subtrait。可以理解为 subtrait 在 supertrait 的约束之上，又多了一套新的约束。这些不同约束的地位是平等的。

约束中同名方法的访问

有的时候多个约束上会定义同名方法，像下面这样：

[复制代码](#)

```

1 trait Shape {
2     fn play(&self) {    // 定义了play()方法
3         println!("1");
4     }
5 }
6 trait Circle : Shape {
7     fn play(&self) {    // 也定义了play()方法
8         println!("2");
9     }
10 }
11 struct A;
12 impl Shape for A {}
13 impl Circle for A {}
14
15 impl A {

```



```

16     fn play(&self) {      // 又直接在A上实现了play()方法
17         println!("3");
18     }
19 }
20
21 fn main() {
22     let a = A;
23     a.play();      // 调用类型A上实现的play()方法
24     <A as Circle>::play(&a); // 调用trait Circle上定义的play()方法
25     <A as Shape>::play(&a);  // 调用trait Shape上定义的play()方法
26 }
27 //输出
28 3
29 2
30 1

```

上面示例展示了两个不同的 trait 定义同名方法，以及在类型自身上再定义同名方法，然后是如何精准地调用到不同的实现的。可以看到，在 Rust 中，同名方法没有被覆盖，能精准地路由过去。

`<A as Circle>::play(&a);` 这种语法，叫做**完全限定语法**，是调用类型上某一个方法的完整路径表达。如果 `impl` 和 `impl trait` 时有同名方法，用这个语法就可以明确区分出来。

用 trait 实现能力配置

trait 提供了寻找方法的范围

Rust 在一个实例上是怎么检查有没有某个方法的呢？

1. 检查有没有直接在这个类型上实现这个方法。
2. 检查有没有在这个类型上实现某个 trait，trait 中有这个方法。

一个类型可能实现了多个 trait，不同的 trait 中各有一套方法，这些不同的方法中可能还会出现同名方法。Rust 在这里采用了一种惰性的机制，由开发者指定在当前的 `mod` 或 `scope` 中使用哪套或哪几套能力。因此，对应地需要开发者手动地将要用到的 trait 引入当前 `scope`。

比如下面这个例子，我们定义两个隔离的模块，并在 module_b 里引入 module_a 中定义的类型 A。

 复制代码

```
1 mod module_a {
2     pub trait Shape {
3         fn play(&self) {
4             println!("1");
5         }
6     }
7
8     pub struct A;
9     impl Shape for A {}
10 }
11
12 mod module_b {
13     use super::module_a::A; // 这里只引入了另一个模块中的类型
14
15     fn doit() {
16         let a = A;
17         a.play();
18     }
19 }
```

报错了，怎么办呢？

```

1 error[E0599]: no method named `play` found for struct `A` in the current scope
2   --> src/lib.rs:17:11
3   |
4 3 |         fn play(&self) {
5   |             ---- the method is available for `A` here
6   ...
7 8 |     pub struct A;
8   |         ----- method `play` not found for this struct
9   ...
10 17 |         a.play();
11   |             ^^^^^ method not found in `A`
12   |
13   = help: items from traits can only be used if the trait is in scope
14 help: the following trait is implemented but not in scope; perhaps add a `use` fo
15   |
16 13 +     use crate::module_a::Shape;

```

引入 trait 就可以了。

```

1 mod module_a {
2     pub trait Shape {
3         fn play(&self) {
4             println!("{}", "1");
5         }
6     }
7
8     pub struct A;
9     impl Shape for A {}
10 }
11
12 mod module_b {
13     use super::module_a::Shape; // 引入这个trait
14     use super::module_a::A;
15
16     fn doit() {
17         let a = A;
18         a.play();
19     }
20 }

```

也就是说，在当前 mod 不引入对应的 trait，你就得不到相应的能力。因此 **Rust 的 trait 需要引入当前 scope 才能使用的方式可以看作是能力配置（Capability Configuration）机制。**

约束可按需配置

有了 trait 这种能力配置机制，我们可以在需要的地方按需加载能力。需要什么能力就引入什么能力（提供对应的约束）。不需要一次性限制过死，比如下面的示例就演示了几种约束组合的可能性。

[复制代码](#)

```
1 trait TraitA {}
2 trait TraitB {}
3 trait TraitC {}
4
5 struct A;
6 struct B;
7 struct C;
8
9 impl TraitA for A {}
10 impl TraitB for A {}
11 impl TraitC for A {} // 对类型A实现了TraitA, TraitB, TraitC
12 impl TraitB for B {}
13 impl TraitC for B {} // 对类型B实现了TraitB, TraitC
14 impl TraitC for C {} // 对类型C实现了TraitC
15
16 // 7个版本的doit() 函数
17 fn doit1<T: TraitA + TraitB + TraitC>(t: T) {}
18 fn doit2<T: TraitA + TraitB>(t: T) {}
19 fn doit3<T: TraitA + TraitC>(t: T) {}
20 fn doit4<T: TraitB + TraitC>(t: T) {}
21 fn doit5<T: TraitA>(t: T) {}
22 fn doit6<T: TraitB>(t: T) {}
23 fn doit7<T: TraitC>(t: T) {}
24
25 fn main() {
26     doit1(A);
27     doit2(A);
28     doit3(A);
29     doit4(A);
30     doit5(A);
31     doit6(A);
32     doit7(A); // A的实例能用在所有7个函数版本中
33 }
```

```
34     doit4(B);
35     doit6(B);
36     doit7(B); // B的实例只能用在3个函数版本中
37
38     doit7(C); // C的实例只能用在1个函数版本中
39 }
```

示例里，A 的实例能用在全部的（7 个）函数版本中，B 的实例只能用在 3 个函数版本中，C 的实例只能用在 1 个函数版本中。

我们再来看一个示例，这个示例演示了如何对带类型参数的结构体在实现方法的时候，按需求施加约束。

[复制代码](#)

```
1 use std::fmt::Display;
2
3 struct Pair<T> {
4     x: T,
5     y: T,
6 }
7
8 impl<T> Pair<T> { // 第一次 impl
9     fn new(x: T, y: T) -> Self {
10         Self { x, y }
11     }
12 }
13
14 impl<T: Display + PartialOrd> Pair<T> { // 第二次 impl
15     fn cmp_display(&self) {
16         if self.x >= self.y {
17             println!("The largest member is x = {}", self.x);
18         } else {
19             println!("The largest member is y = {}", self.y);
20         }
21     }
22 }
```

这个示例中，我们对类型 `Pair<T>` 做了两次 `impl`。可以看到，第二次 `impl` 时，添加了约束 `T: Display + PartialOrd`。

因为我们 `cmd_display` 方法需要用到打印能力和元素比大小排序的能力，所以对类型参数 `T` 施加了 `Display` 和 `PartialOrd` 两种约束（两种能力）。而对于 `new` 函数来说，它不需要这些能力，因此 `impl` 的时候就可以不施加这些约束。我们前面讲过，`Rust` 中对类型是可以多次 `impl` 的。

关于 `trait`，你还要了解什么？

孤儿规则

为了不导致混乱，`Rust` 要求在一个模块中，如果要对一个类型实现某个 `trait`，这个类型和这个 `trait` 其中必须有一个是在当前模块中定义的。比如下面这两种情况都是可以的。

情况 1：

```
1 use std::fmt::Display;
2
3 struct A;
4 impl Display for A {}
```

 复制代码

情况 2：

```
1 trait TraitA {}
2 impl TraitA for u32 {}
```

 复制代码

但是下面这样不可以，会编译报错。

```
1 use std::fmt::Display;
2
3 impl Display for u32 {}
```

 复制代码

```

1 error[E0117]: only traits defined in the current crate can be implemented for pri
2 --> src/lib.rs:3:1
3 |
4 3 | impl Display for u32 {}
5 | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
6 | |                                     |
7 | |                                     `u32` is not defined in the current crate
8 | impl doesn't use only types from inside the current crate
9 |
10 = note: define and implement a trait or new type instead

```

因为我们想给一个外部类型实现一个外部 trait，这是不允许的。Rustc 小助手提示我们，如果实在想用的话，可以用 Newtype 模式。

比如像下面这样：

```

1 use std::fmt::Display;
2
3 struct MyU32(u32);    // 用 MyU32 代替 u32
4
5 impl Display for MyU32 {
6     // 请实现完整
7 }
8
9 impl MyU32 {
10     fn get(&self) -> u32 { // 需要定义一个获取真实数据的方法
11         self.0
12     }
13 }

```

Blanket Implementation

Blanket Implementation 又叫做统一实现。

方式如下：

 复制代码

```

1 trait TraitA {}
2 trait TraitB {}
3
4 impl<T: TraitB> TraitA for T {} // 这里直接对T进行实现TraitA

```

统一实现后，就不要对某个具体的类型再实现一次了。因为同一个 trait 只能实现一次到某个类型上。这个不像是类型做 impl，可以实现多次（函数名要不冲突）。

比如：

 复制代码

```

1 trait TraitA {}
2 trait TraitB {}
3
4 impl<T: TraitB> TraitA for T {}
5
6 impl TraitB for u32 {}
7 impl TraitA for u32 {}

```

这样就会报错。

 复制代码

```

1 error[E0119]: conflicting implementations of trait `TraitA` for type `u32`
2   --> src/lib.rs:10:1
3   |
4 6 |   impl<T: TraitB> TraitA for T {}
5   |   ----- first implementation here
6   ...
7 10 |   impl TraitA for u32 {}
8   |   ^^^^^^^^^^^^^^^^^^^^^^^^^^^ conflicting implementation for `u32`
9

```

我们修改一下，这样就不会报错了。

 复制代码


```
1 trait TraitA {}
3 trait TraitB {}
4
5 impl<T: TraitB> TraitA for T {}
6
   impl TraitA for u32 {}
```

因为 u32 并没有被 TraitB 约束，所以它不满足第 4 行的 blanket implementation。因此就不算重复实现。

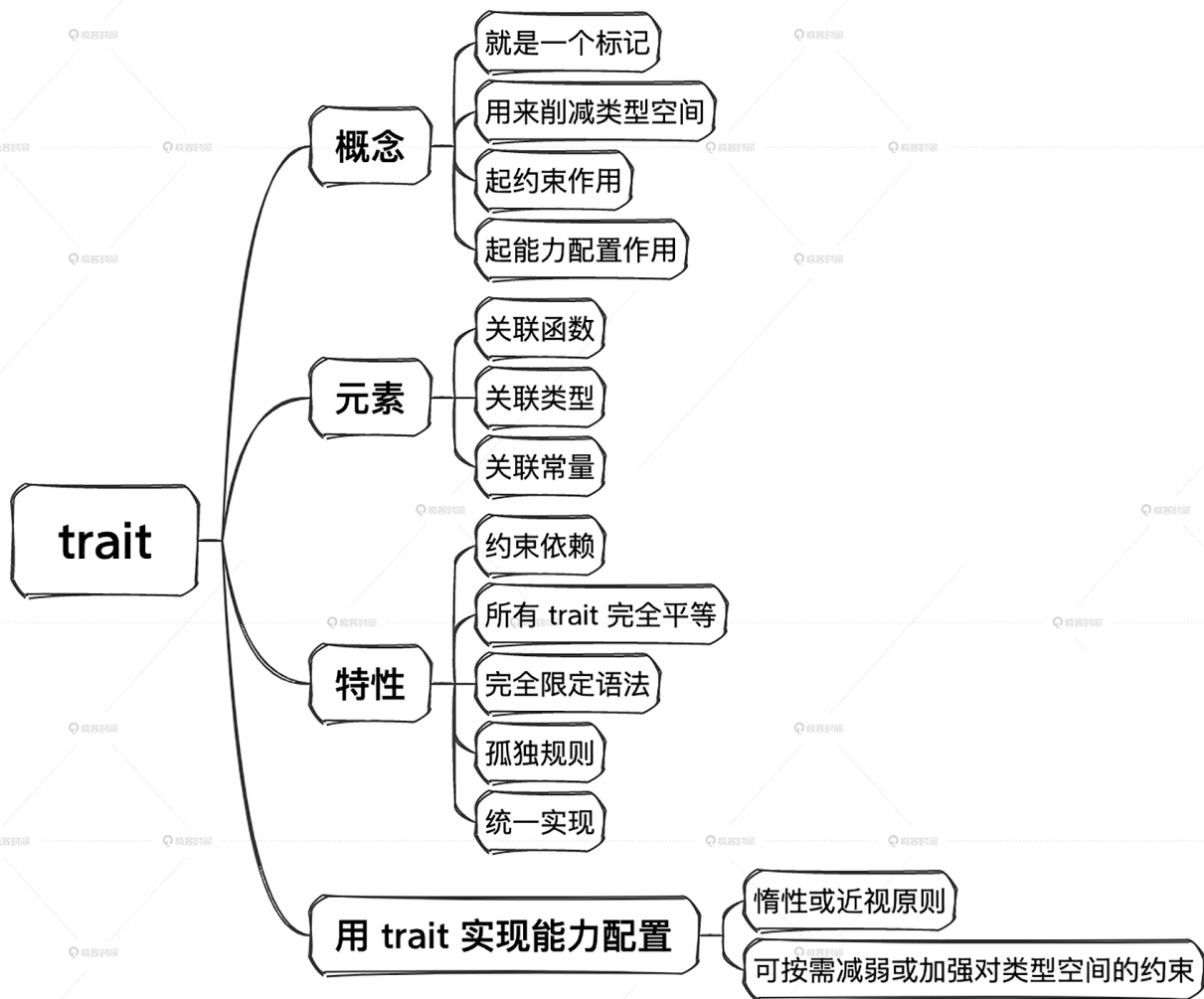
小结

认真学完了这节课的内容之后，你有没有被震撼到？这完全就是一种全新的思维体系，和之前我们熟悉的 OOP 等方式完全不同了。Rust 中 trait 的概念本身非常简单，但用法又极其灵活。

trait 的引入是为了对泛型的类型空间进行约束，进入约束的同时也就提供了能力，约束与能力是一体两面。trait 中可以包含关联函数、关联类型和关联常量。其中关联类型的理解难度较大，但是其模式也就那么固定的几种，多花点时间熟悉一般不会有问题。

trait 定义好后，可以作为代码与代码之间、代码与开发者之间、开发者与开发者之间的强制性法律协议而存在，而这个法律的仲裁者就是 Rustc 编译器。

可以说，**Rust 是一门面向约束编程的语言**。面向约束是 Rust 中非常独特的设计，也是 Rust 的灵魂。简单地把 trait 当作其他语言中的 class 或 interface 去理解使用，是非常有害的。



思考题

如果你学习或者了解过 Java、C++ 等面向对象语言的话，可以聊一聊 trait 的依赖和 OOP 继承的区别在哪里。

欢迎你把思考后的结果分享到评论区，也欢迎你把这节课的内容分享给对 Rust 感兴趣的朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

最新 精选



Mike Tang 置顶

2023-11-09 来自重庆

神说，众生平等。Rust说，众trait平等。

共 1 条评论 >

👍 6



下雨天

2023-11-08 来自湖北

trait 的依赖：小明要听从数学老师，语文老师，英语老师的话。老师之间是平等关系，多个依赖平等，最小依赖选择自己喜欢滴功能。

OOP 继承：小明要听他爸，他爷爷，他曾祖父的话。继承之间存在父子关系，继承过来一堆破属性和方法，也许根本不是自己想要滴，还要负重前行。

作者回复：太形象了，10000个赞

共 3 条评论 >

👍 15



superggn

2023-12-15 来自北京

笔记

一般我们说关联类型 / associated types 的时候，说的是 trait 底下的 type

关联类型就是跟着 trait 走的泛型

至于啥时候关联类型要具化 / 单态化 / 具体写明白是啥？

得在 impl trait for typeA 的时候写清楚

一般说 A 有 trait bound，意思就是这个 A 必须实现某个 trait

我们不仅可以搞 trait bound, 有 trait bound 之后在 where clause 里还可以搞 trait type bound:

```
`T: trait A, T::typeB: Debug + PartialEq`
```

为啥调用 trait 方法还需要 use trait?

因为 rust 不会大海捞针遍历所有 trait 找方法, 你得先指定在哪儿找

作者回复: 非常 🌈 棒



1



\$侯

2023-11-17 来自浙江

既然关联类型作为占位符, 那为什么这样会报错呢, 按我理解理占位符应该不需要事先声明

```
pub trait Sport {
```

```
    type SportType;
```

```
    fn play(&self, st: SportType);
```

```
}
```

```
fn main() {}
```

```
error[E0412]: cannot find type `SportType` in this scope
```

```
--> src\main.rs:4:24
```

```
|
```

```
4 |     fn play(&self, st: SportType);
```

```
|
```

```
^^^^^^^^ help: you might have meant to use the associated type: `
```

```
Self::SportType`
```

作者回复: 他提示了, 用 Self::SportType, 完整路径。我发现我那个例子也漏了, 我加上。



1



cluski □

2023-11-08 来自江苏

个人理解，trait很像Java中的interface。Java的interface可以作为某种能力的抽象，并且在泛型的使用中，可以起到限制的作用。

Java、C++等OOP语言中的继承个人感觉更多是在强调is-a这个概念。例如，男人是一个人，鸽子是一个鸟这类。与Rust的trait更加强调的一种能力和约束。

作者回复: 对的。rust的trait将模型从oop的等级森严的牢房里面解救出来了



1



weineel

2024-01-01 来自江苏

在语法层面实践了组合优于继承。

作者回复: 对的对的.



千回百转无劫山

2023-12-27 来自北京

从python过来的，只能说打开了新世界的大门

作者回复: rust不给开发者设置天花板



superggn

2023-12-15 来自北京

思考题（我从 python 来的， cpp 和 java 没学过）

OOP

class 里有各种方法，搞个子类就可以对基类进行各种修改

定义个 class, class 里有各种方法（包括 static method 啥的）

大部分情况是一个树形结构

面向 trait bound 编程

trait 类似于 OOP 里搞一个标准基类，然后来回继承，不同点在于更灵活

或者类似于 python3 里的 mixin（这玩意我没怎么玩过，不熟，但看过同事写的代码），搞个 must implement 的 abstract method 感觉也差不多

引用来引用去，飞来飞去~

这种思考题没啥感觉，反正我也不会写，随便叨叨，等全都刷一遍之后可能会有别的感想

作者回复：最好不要通过继承那个思维来理解，对的，先刷完一遍再回头来看。



superggn

2023-12-15 来自北京

我记得 trait 底下定义的函数里，有 self 参数的叫方法（method, called on instance），没 self 参数的才叫 关联函数（associated function, called on type)?

=====

刚才查了下，duckduckgo 搜出来的 rust associated function 跟我记得一样，区分 method 和 associated function，但 Rust By Practice (RBP) 和 Rust By Example (RBE) 说的不一样...

RBP

<https://practice.rs/method.html>

=> "All functions defined within an impl block are called associated functions because they're associated with the type named after the impl." 所有在 impl 底下的函数都能叫做 associated functions

RBE

<https://doc.rust-lang.org/rust-by-example/fn/methods.html>

=> "Some functions are connected to a particular type. These come in two forms: associated functions, and methods."

作者回复: 我是根据rust reference的说法来的，其实意思都差不多，这个局部可以互换理解。两种说法都是对的

共 2 条评论 >



plh

2023-11-29 来自四川

老师你好: 原文 [因为同一个 trait 只能实现一次到某个类型上。] 这个 "某个类型" 怎么理解? 比如: 标准库上有 Option 上面 就有这么3个方法: 感觉这个有点迷惑?

```
impl<T> IntoIterator for Option<T>
impl<'a, T> IntoIterator for &'a Option<T>
impl<'a, T> IntoIterator for &'a mut Option<T>
```

作者回复: 二三是引用类型的引用类型进行实现，不是类型本身。

